306X 蓝牙 PAN 接入小智 AI

版本号: V1.0
日期: 2025.12

FreqChip Confi

FREQCHIP
富芮坤

修订版本

版本	日期	更新内容	修订人员
V1.0	2026.04.13	首版	刘浩

FreqChip Confidential

目录

1. 概述 1
2. 系统移植说明 ..... 错误!未定义书签。
2.1 工程准备 ..... 错误!未定义书签。
2.2 WebSoket 任务说明 ..... 错误!未定义书签。
2.3 小智 AI 数据处理流程 ..... 错误!未定义书签。
2.4 音频编解码任务 ..... 错误!未定义书签。
2.5 工程配置注意事项 ..... 错误!未定义书签。
联系方式 10

FreqChip Confidential

1. 概述

本文档描述基于 BT PAN 的网络接入方式，并结合 WebSocket 实现与小智 AI 服务器的音频交互流程，包括工程移植、网络连接、音频编解码及状态机说明。

FreqChip Confidential

2. 系统移植说明

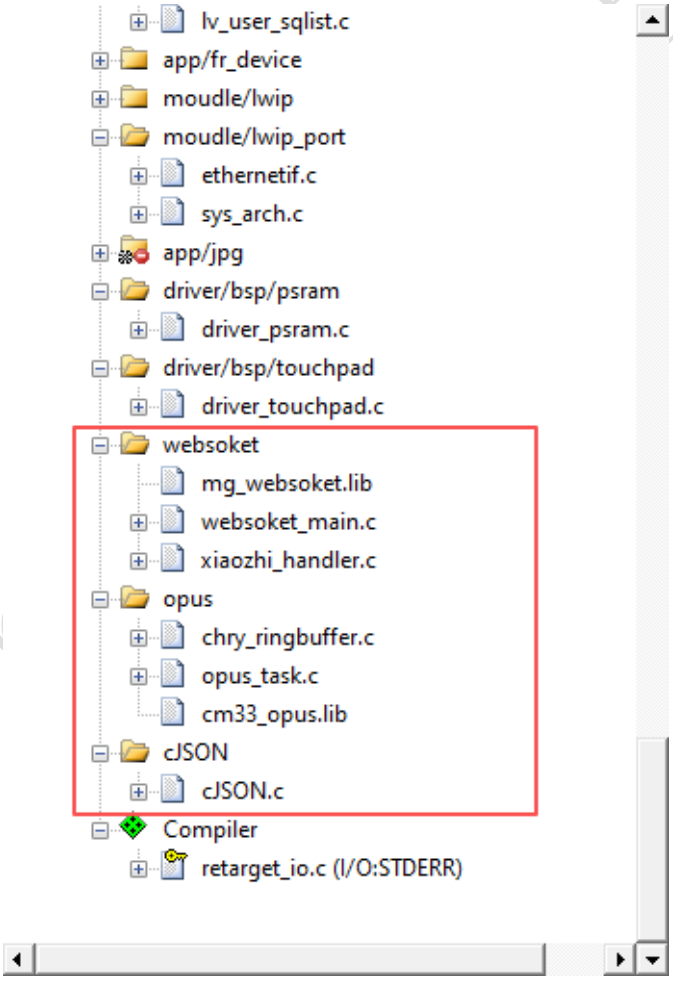
2.1 工程准备

1、 准备一个带 bt 功能的工程, 打开工程中的 app_config.h 文件, 配置 BTDM_STACK_ENABLE_PAN=1。

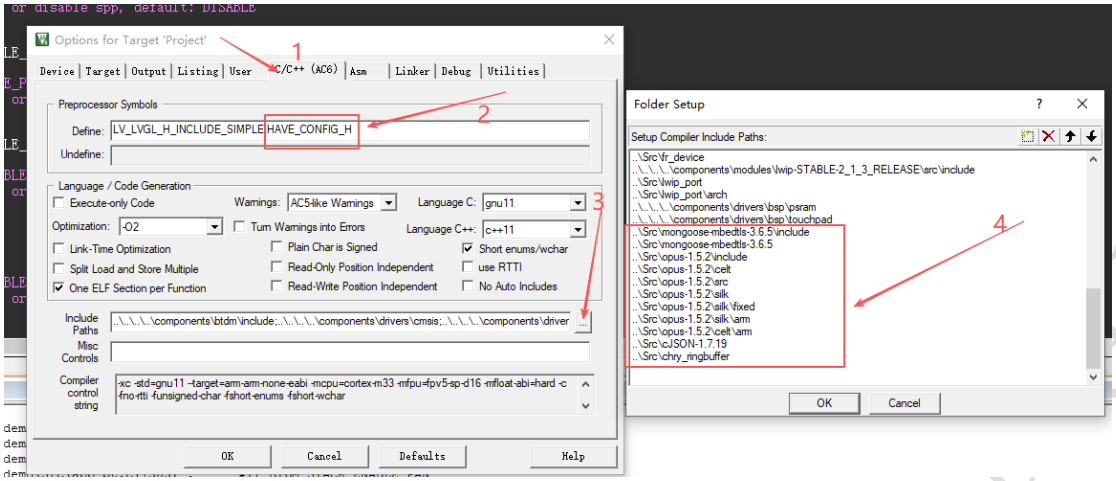
2、 复制如下文件到工程目录的 Src 文件夹下，这里以 lvgl_demo 工程为例



3、 依次将下面的这些文件添加到工程里面

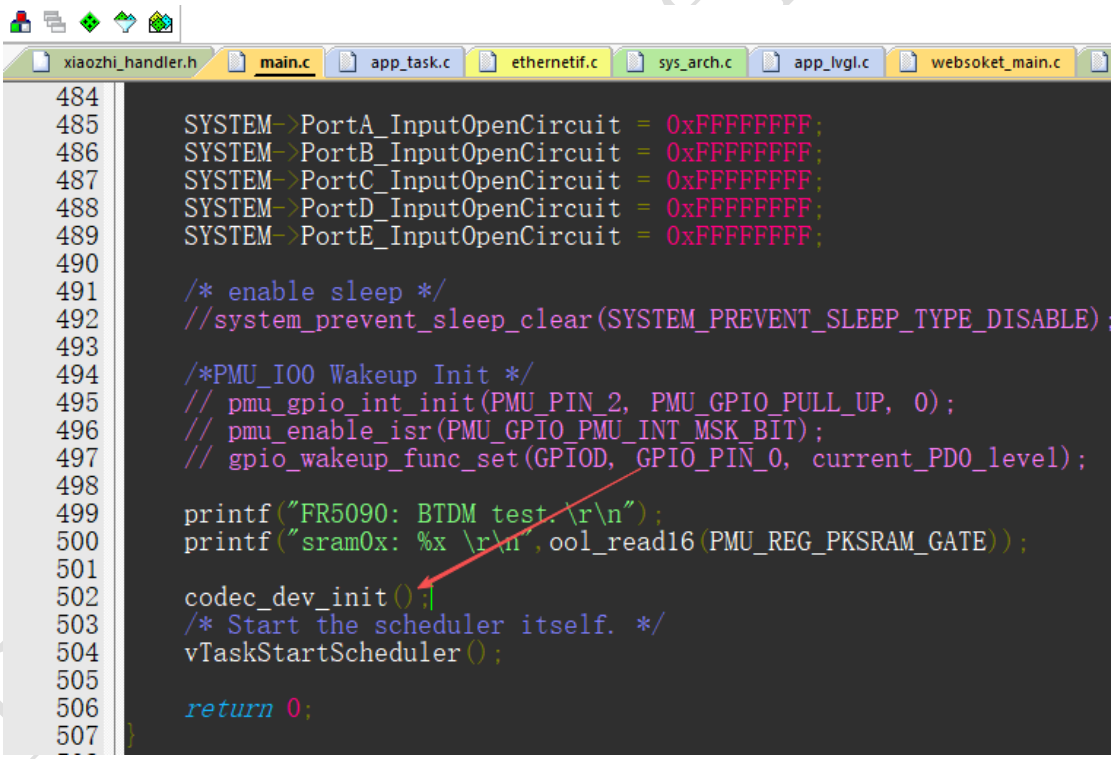


4、 配置工程宏和头文件路径如下图

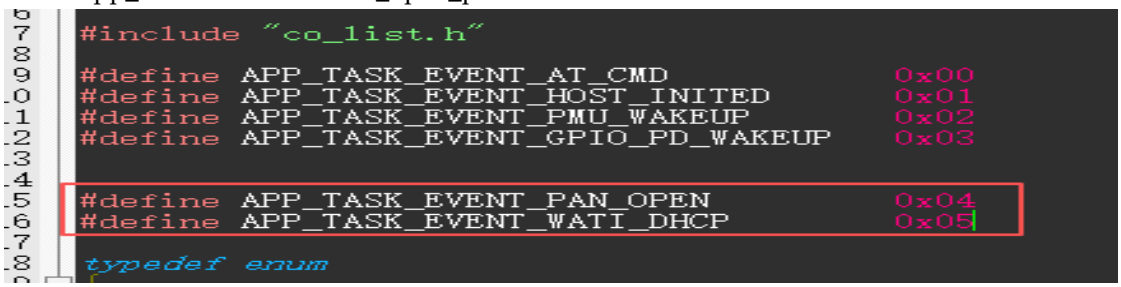


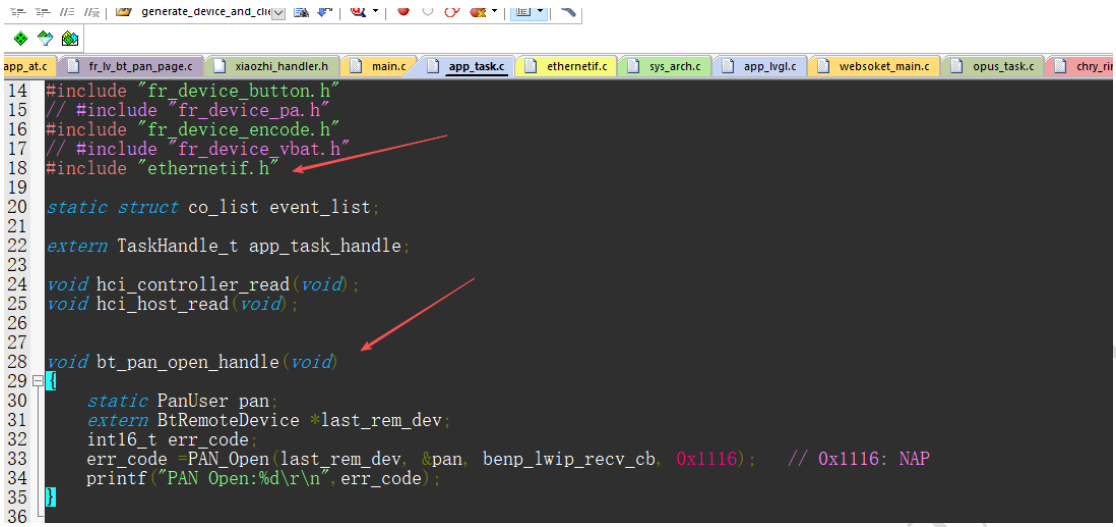
5、 替换掉 lwip_port 文件夹下面的 ethernetif.c，sys_arch.c 文件

6、 初始化音频编解码任务和外设驱动程序，如下图



7、 在 app_task.c 里面封装好 bt_open_pan 函数接口，然后添加以下两个事件处理。





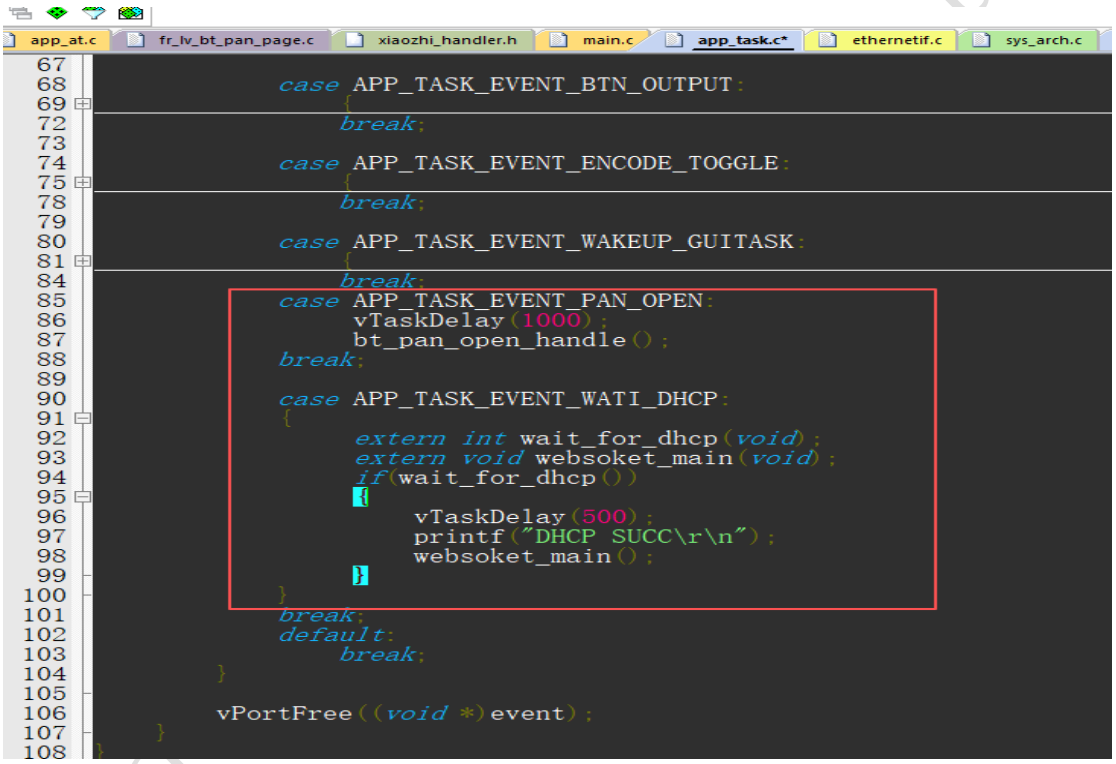
```c
#include "fr_device_button.h"
// #include "fr_device_pa.h"
#include "fr_device_encode.h"
// #include "fr_device_vbat.h"
#include "ethernetif.h"

static struct co_list event_list;

extern TaskHandle_t app_task_handle;

void hci_controller_read(void);
void hci_host_read(void);


void bt_pan_open_handle(void)
{
    static PanUser pan;
    extern BtRemoteDevice *last_rem_dev;
    int16_t err_code;
    err_code =PAN_Open(last_rem_dev, &pan, benp_lwip_recv_cb, 0x1116);   // 0x1116: NAP
    printf("PAN Open:%d\r\n",err_code);
}
```



```c
            case APP_TASK_EVENT_BTN_OUTPUT:
                {
                }
                break;

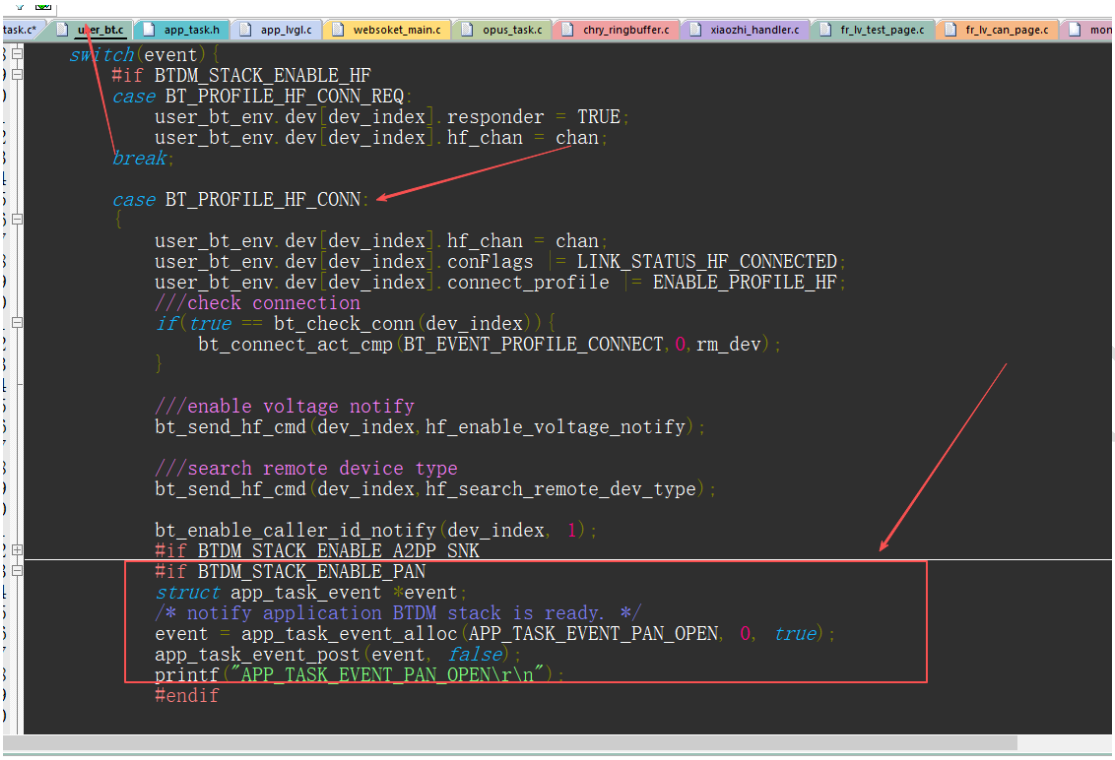
            case APP_TASK_EVENT_ENCODE_TOGGLE:
                {
                }
                break;

            case APP_TASK_EVENT_WAKEUP_GUITASK:
                {
                }
                break;
            case APP_TASK_EVENT_PAN_OPEN:
                vTaskDelay(1000);
                bt_pan_open_handle();
            break;

            case APP_TASK_EVENT_WATI_DHCP:
            {
                extern int wait_for_dhcp(void);
                extern void websoket_main(void);
                if(wait_for_dhcp())
                {
                    vTaskDelay(500);
                    printf("DHCP SUCC\r\n");
                    websoket_main();
                }
            }
            break;
            default:
                break;
        }

        vPortFree((void *)event);
    }
}
```

8、 在 bt 连接事件里面添加发送开启 PAN 的事件到 app_task 里面去处理，在 PAN 连接事件里面发送等待 DHCP 完成处理，完成后创建 websoket 任务去连接小智 ai 服务器，如下图。



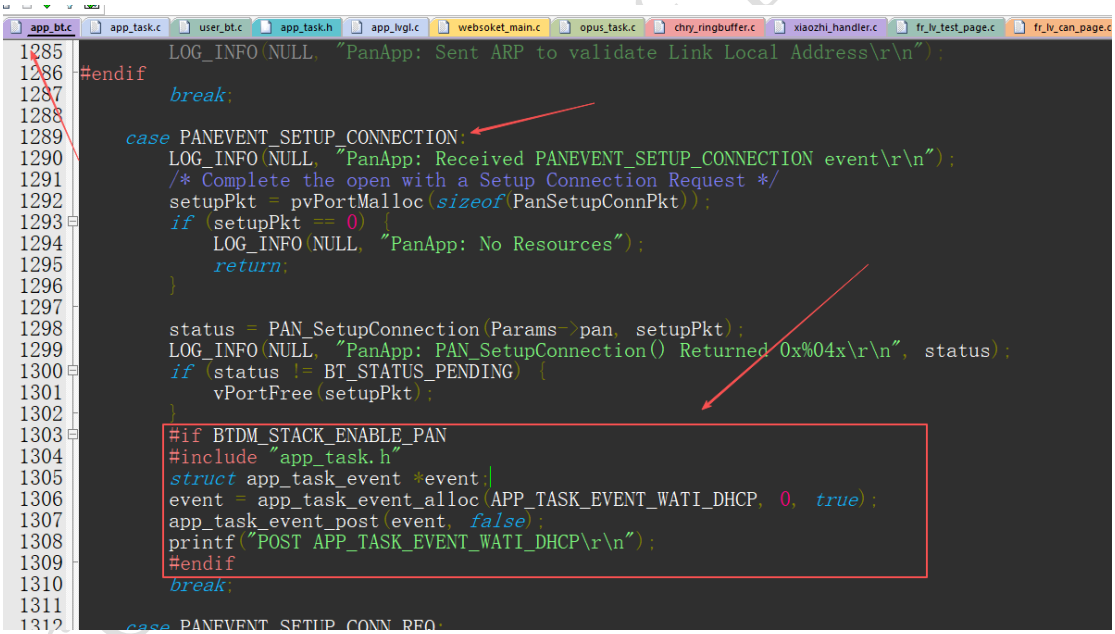
```
    switch(event){
        #if BTDM_STACK_ENABLE_HF
        case BT_PROFILE_HF_CONN_REQ:
            user_bt_env.dev[dev_index].responder = TRUE;
            user_bt_env.dev[dev_index].hf_chan = chan;
        break;

        case BT_PROFILE_HF_CONN:
        {
            user_bt_env.dev[dev_index].hf_chan = chan;
            user_bt_env.dev[dev_index].conFlags |= LINK_STATUS_HF_CONNECTED;
            user_bt_env.dev[dev_index].connect_profile |= ENABLE_PROFILE_HF;
            ///check connection
            if(true == bt_check_conn(dev_index)){
                bt_connect_act_cmp(BT_EVENT_PROFILE_CONNECT,0,rm_dev);
            }

            ///enable voltage notify
            bt_send_hf_cmd(dev_index,hf_enable_voltage_notify);

            ///search remote device type
            bt_send_hf_cmd(dev_index,hf_search_remote_dev_type);

            bt_enable_caller_id_notify(dev_index, 1);
            #if BTDM_STACK_ENABLE_A2DP_SNK
            #if BTDM_STACK_ENABLE_PAN
            struct app_task_event *event;
            /* notify application BTDM stack is ready. */
            event = app_task_event_alloc(APP_TASK_EVENT_PAN_OPEN, 0, true);
            app_task_event_post(event, false);
            printf("APP_TASK_EVENT_PAN_OPEN\r\n");
            #endif
```



```
1285            LOG_INFO(NULL, "PanApp: Sent ARP to validate Link Local Address\r\n");
1286    #endif
1287            break;
1288
1289        case PANEVENT_SETUP_CONNECTION:
1290            LOG_INFO(NULL, "PanApp: Received PANEVENT_SETUP_CONNECTION event\r\n");
1291            /* Complete the open with a Setup Connection Request */
1292            setupPkt = pvPortMalloc(sizeof(PanSetupConnPkt));
1293            if (setupPkt == 0) {
1294                LOG_INFO(NULL, "PanApp: No Resources");
1295                return;
1296            }
1297
1298            status = PAN_SetupConnection(Params->pan, setupPkt);
1299            LOG_INFO(NULL, "PanApp: PAN_SetupConnection() Returned 0x%04x\r\n", status);
1300            if (status != BT_STATUS_PENDING) {
1301                vPortFree(setupPkt);
1302            }
1303            #if BTDM_STACK_ENABLE_PAN
1304            #include "app_task.h"
1305            struct app_task_event *event;
1306            event = app_task_event_alloc(APP_TASK_EVENT_WATI_DHCP, 0, true);
1307            app_task_event_post(event, false);
1308            printf("POST APP_TASK_EVENT_WATI_DHCP\r\n");
1309            #endif
1310            break;
1311
1312        case PANEVENT_SETUP_CONN_REQ:
```

2.2 WebSoket 任务说明

Websoket_task 任务原型代码如下，主要通过 mg 库实现 WSS 客户端和音频发送状态机。任务里循环轮询网络事件，通过 IDLE、CONNECTING、CONNECTED、STREAMING、FINISHING 几个状态来控制连接和流程：空闲时自动发起带鉴权信息的 WSS 连接；连接成功后进入已连接状态。所有网络事件都在 ws_xiaozhi_handler 回调中统一处理，包括 TLS 握手、WebSocket 建立、消息收发以及异常断开。音频发送由按键触发，按下发送 START 并持续从 Opus 链表里面取数据通过 WebSocket 发送，松开后把剩余音频发完，再发送 STOP 消息。同时加入了自动重连机制：服务器每间隔 1 分钟会断开客户端或出现异常时会清空连接状态并回到空闲态，在主循环中满足条件后自动重新连接。

```
void websocket_task(void *arg)
{
//    struct mg_mgr mgr;
    mg_mgr_init(&mgr);
    mg_log_set(MG_LL_DEBUG);
    #ifndef DEVICE_ID
    uint8_t bt_addr[6];
    uint8_t id_length = 5;
    bt_addr[0]=0xfa;
    system_get_unique_ID(&bt_addr[1], &id_length);
    generate_device_and_client_id(bt_addr);
    #endif

    TickType_t last_tick_60ms = 0;
    while (1)
    {
        mg_mgr_poll(&mgr, 10);
        vTaskDelay(pdMS_TO_TICKS(10));

        switch (g_ws_state) {
        /*---------------- 空闲：自动连接 ----------------*/
        case STATE_IDLE:
            if(!g_xiaozhi_param.is_connected && (xTaskGetTickCount() - last_recv_time >= 500))
            {
                wss_start_connect(&mgr);
            }
            break;

        /*---------------- 等待连接成功 ----------------*/
        case STATE_CONNECTING:
            if (g_xiaozhi_param.is_connected) {
                g_ws_state = STATE_CONNECTED;
            }
            break;


        case STATE_CONNECTED:
            if (xTaskGetTickCount() - last_tick_60ms >= 60) {
                last_tick_60ms = xTaskGetTickCount();

                if (g_key_down) {
                    MG_INFO(("Key pressed -> START"));
                    xiaozhi_send_msg_start();
                    g_ws_state = STATE_STREAMING;
                }
            }
            break;

        /*---------------- 正在发送音频 ----------------*/
        case STATE_STREAMING:
            if (!g_key_down) {
```

app_task.c | user_bt.c | app_btdm.c | app_lvgl.c | websoket_main.c* | opus_task.c | chry_ringbuffer.c

```c
242
243 static void ws_xiaozhi_handler(struct mg_connection *c, int ev, void *ev_data) {
244     switch (ev) {
245
246     case MG_EV_RESOLVE:
247         MG_INFO(("The domain name is being resolved..."));
248         break;
249
250     case MG_EV_CONNECT:
251         MG_INFO(("TCP is connected (WSS handshake is about to start)..."));
252         struct mg_tls_opts opts = {
253             .name = mg_url_host(XZ_WSS_URL),
254             .skip_verification = 1      // 相当于 websocketpp 的 verify_none
255         };
256         mg_tls_init(c, &opts);
257         MG_INFO(("TLS init"));
258         break;
259
260     case MG_EV_TLS_HS:
261         MG_INFO(("TLS handshake completed"));
262         break;
263
264     case MG_EV_WS_OPEN:
265         MG_INFO(("WebSocket MG_EV_WS_OPEN"));
266         mg_conn_sta = true;
267         g_xiaozhi_param.ws_conn = g_ws_conn;
268         g_xiaozhi_param.is_connected = 1;
269         g_ws_state = STATE_CONNECTED;
270         last_audio_send_time++;
271         mg_ws_send(c,hello_message, strlen(hello_message), WEBSOCKET_OP_TEXT);
272         break;
273
274     case MG_EV_WS_MSG: {
275         struct mg_ws_message *wm = (struct mg_ws_message *) ev_data;
276         //MG_INFO(("msg: %.*s", (int)wm->data.len, wm->data.buf));
277         last_recv_time = xTaskGetTickCount();    // 更新接收时间戳
278         parse_ws_message(wm->data.buf, wm->data.len);
279
280         break;
281     }
282     case MG_EV_ERROR:
283     case MG_EV_CLOSE:
284         g_ws_conn = NULL;
285         mg_conn_sta = false;
286         g_xiaozhi_param.is_connected = 0;
```

app_task.c | user_bt.c | app_btdm.c | app_lvgl.c | websoket_main.c* | opus_task.c | chry_ringbuffer.c

```c
242
243 static void ws_xiaozhi_handler(struct mg_connection *c, int ev, void *ev_data) {
244     switch (ev) {
245
246     case MG_EV_RESOLVE:
247         MG_INFO(("The domain name is being resolved..."));
248         break;
249
250     case MG_EV_CONNECT:
251         MG_INFO(("TCP is connected (WSS handshake is about to start)..."));
252         struct mg_tls_opts opts = {
253             .name = mg_url_host(XZ_WSS_URL),
254             .skip_verification = 1      // 相当于 websocketpp 的 verify_none
255         };
256         mg_tls_init(c, &opts);
257         MG_INFO(("TLS init"));
258         break;
259
260     case MG_EV_TLS_HS:
261         MG_INFO(("TLS handshake completed"));
262         break;
263
264     case MG_EV_WS_OPEN:
265         MG_INFO(("WebSocket MG_EV_WS_OPEN"));
266         mg_conn_sta = true;
267         g_xiaozhi_param.ws_conn = g_ws_conn;
268         g_xiaozhi_param.is_connected = 1;
269         g_ws_state = STATE_CONNECTED;
270         last_audio_send_time++;
271         mg_ws_send(c,hello_message, strlen(hello_message), WEBSOCKET_OP_TEXT);
272         break;
273
274     case MG_EV_WS_MSG: {
275         struct mg_ws_message *wm = (struct mg_ws_message *) ev_data;
276         //MG_INFO(("msg: %.*s", (int)wm->data.len, wm->data.buf));
277         last_recv_time = xTaskGetTickCount();    // 更新接收时间戳
278         parse_ws_message(wm->data.buf, wm->data.len);
279
280         break;
281     }
282     case MG_EV_ERROR:
283     case MG_EV_CLOSE:
284         g_ws_conn = NULL;
285         mg_conn_sta = false;
286         g_xiaozhi_param.is_connected = 0;
```

2.3 小智 AI 数据处理流程

小智消息处理如下，系统通过 WebSocket 接收服务端下发的数据，并对消息进行统一入口解析，根据数据格式区分为 JSON 控制消息与二进制音频流数据两类：当检测为 JSON 格式时，调用 cJSON 进行解析，提取 type、session_id 及 audio_params 等字段，用于完成会话管理、音频参数配置（如采样率、帧长）以及 TTS 文本等控制信息的处理；当检测为二进制数据时，则认为其为 Opus 编码音频流，不做额外解析，直接推入解码帧队列（如链表或环形缓冲区），由独立的音频解码线程进行异步处理与播放。该设计实现了控制信令与音频数据的解耦，保证了音频链路的实时性与系统整体的可扩展性，代码如下图。

```
void handle_json(const char *buf, size_t len) {
    //printf("[JSON] %.*s\n", (int)len, buf);
    xiaozhi_msg_parse(buf,len);
}

void handle_binary(const uint8_t *buf, size_t len) {
    printf("[BIN] recv %u bytes\r\n", (unsigned)len);
    extern void opus_decode_frame_push(const uint8_t *buf, uint16_t length);
    opus_decode_frame_push(buf,len);
}

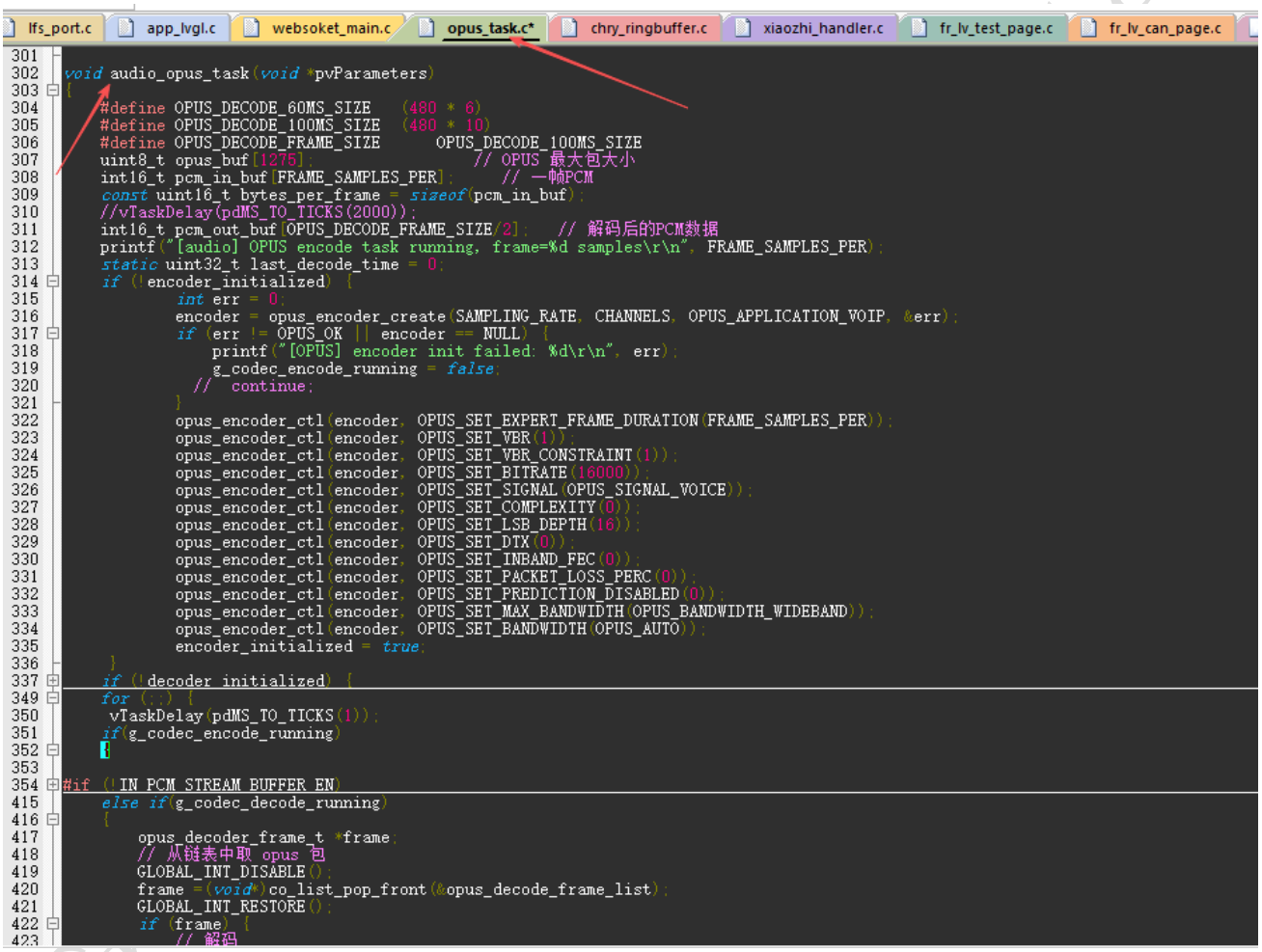
void parse_ws_message(const char *buf, size_t len)
{
    if (is_json(buf, len)) {
        handle_json(buf, len);
    } else {
        handle_binary((uint8_t*)buf, len);
    }
}
```

```
void xiaozhi_msg_parse(const char *data, uint16_t len)
{
    cJSON *item = NULL;
    cJSON *root = NULL;
    root = cJSON_Parse(data);
    if (!root)
    {
        printf("Error before: [%s]\n", cJSON_GetErrorPtr());
        return;
    }
    char *type = cJSON_GetObjectItem(root, "type")->valuestring;
     printf("type = %s\n", type);
    if (strcmp(type, "hello") == 0)
    {
        char *session_id = cJSON_GetObjectItem(root, "session_id")->valuestring;
        printf("session_id = %s\n", session_id);
        cJSON *audio_param = cJSON_GetObjectItem(root, "audio_params");
        char *sample_rate = cJSON_GetObjectItem(audio_param, "sample_rate")->valuestring;
        char *duration = cJSON_GetObjectItem(audio_param, "duration")->valuestring;
        g_xiaozhi_param.audio_sample_rate = atoi(sample_rate);
        g_xiaozhi_param.audio_frame_duration = atoi(duration);
        strncpy((char*)g_xiaozhi_param.session_id, session_id, 9);
        g_xiaozhi_param.work_state = 0;


    }
    else if (strcmp(type, "goodbye") == 0)
```

2.4 音频编解码任务说明

audio_opus_task 是系统中的音频处理任务，主要用于完成 **PCM 音频与 Opus 编码数据之间的双向转换**。任务启动后首先初始化 Opus 编码器与解码器，并根据系统状态在“编码模式”和“解码模式”之间切换运行：在编码模式下，从输入环形缓冲区读取 PCM 数据，按帧进行 Opus 编码，并将编码后的数据封装后写入发送链表；在解码模式下，从音频数据接收链表中获取 Opus 数据帧，进行解码后输出为 PCM 数据，并写入播放缓冲区供音频输出模块使用。任务通过环形缓冲区与链表机制实现数据流解耦，结合临界区保护保证线程安全，同时通过状态标志控制编解码流程的启停。



2.5 工程配置注意事项

1、 wss 加密连接和 opus 编解码任务对堆的占用很高，configTOTAL_HEAP_SIZE ≥ 300KB

2、不要修改修改 lwip 和 websoket 的任务优先级

3、音频 PCM 流缓存使用 PSRAM，注意内存地址不要与其他应用层代码冲突

4、非加密连接需配置 MG_TLS_NONE

联系方式

欢迎大家针对富芮坤产品和文档提出建议。

反馈：doc@freqchip.com

网站：www.freqchip.com

销售：sales@freqchip.com

电话：+86-21-5027-0080

FreqChip Confidential